Introduction to Data Mining, Spring 2026
HW1

Due date: 04/09/2026 (11:59pm)

Submission Instructions: Use eTL to submit your homework. Your submission should be a single zipped tar file named `StudentID_hwX.zip`. For example, name the file `202600000_hw1.zip` if your student ID is 2026-00000 and it is for homework #1. Use only the following extensions: `.tar`, `.gz`, `.zip`, or `.tar.gz`.

Your zip file should contain **five** files; one PDF file for writeup answers (`hw1.pdf`), three python files (`hw1_1.py`, `hw1_2_b.py`,`hw1_3_b.py`, and `hw1_3_c.py`), and the Ethics Oath pdf file. Before zipping your files, create a directory named `StudentID_hwX` and place your files inside this directory. If you violate any of the file naming or extension format requirements, *1 point will be deducted* from your total score per mistake.

Submitting Writeup: Prepare your answers to the homework questions in a single PDF file. You can use the following template. Please write as succinctly as possible.

Submitting Code: Each problem may include a programming component. Place all the code for each problem into a single file. You are allowed to import only the following libraries: `pyspark`, `numpy`, `re`, `sys`, `math`, `csv`, and `os`.[1] *No other libraries are permitted, including standard python library such as* `collections`. Good coding style, including the use of comments, will be one of the grading criteria. Ensure your code is well-structured and includes descriptive comments.

Ethics Oath: For every homework submission, please fill out and submit the PDF version of this document pledging your honor that you did not violate any ethics rules required by this course and SNU. You can either scan a printed version into a PDF file or convert the filled-out Word document with your signature into a PDF file.

Use of AI Tools: The use of AI tools for programming is **not recommended** in this course. Nevertheless, if you use any AI tool in completing this homework, you **must disclose it in writeup**. At the beginning of your writeup, briefly describe which AI tool(s) you used, how you used them, and what assistance they provided. There will be **no point deduction** for the use of AI tools, as long as the disclosure is complete and truthful.

[1] You can use `time` library to check the program's elapsed time.

Discussions with other students are permitted and encouraged. However, when it comes time to write your solution, such discussions (except with the course staff) are no longer appropriate; you must write your own solutions independently. If you received any help, you must specify at the top of your written homework the individuals from whom you received help and the nature of the help received. *Do not, under any circumstances, copy another person's solution.* All submissions are checked for plagiarism, including the ones submitted in previous years of this course, and any violations are taken seriously.

1 MapReduce and Spark (30 pts)

We will find potential friends in a social network using Spark.

Social networks like Facebook, Twitter, and LiveJournal have users who are connected as friends in the form of a graph. In this problem, you will use a real dataset containing a LiveJournal friends graph to discover potential friends who have many mutual friends.

Download `soc-LiveJournal1Adj.txt`[2] from eTL and use it as a dataset. Each line of the dataset is in the following format:

```
<User><TAB><Friends>
```

where `<User>` is an integer ID corresponding to a user, `<TAB>` is the tab character, and `<Friends>` is a comma-separated list of IDs of friends.

We are interested in finding pairs of users who are 1) not friends with each other, but 2) have many common friends. For example, if A is a friend of B, B is a friend of C, but A is not a friend of C, then we are interested in the pair (A, C), which has one common friend B.

The output should be in this format:

```
<User><TAB><User><TAB><Count>
```

Example (one line):
`18739<TAB>18740<TAB>100`

where the first user ID is always smaller than the second user ID, and `<Count>` contains the number of mutual friends between the two users. The lines must be sorted by their counts in descending order. In case there are ties in counts, the lines are sorted by the first user ID integer in ascending order and then the second user ID integer in ascending order.

Here are some hints:

- The Spark Programming Guide may be useful: https://spark.apache.org/docs/latest/rdd-programming-guide.html.
- Before running on the entire dataset, debug your code on a small test dataset.
- If you run out of memory running Spark, try adding the flag --driver-memory 8G (on a Spark cluster, --executor-memory 8G). Also consider using a machine in the DM01 server.
- Although this problem requires thought, the code itself does not have to be long.

Please **use command-line** arguments to obtain the file path of the dataset. (Do not fix the path in your code.) For example, run:

[2] This dataset is from the Stanford Large Network Dataset Collection.

```
python hw1_1.py YourPathTo/soc-LiveJournal1Adj.txt
```

Please submit the following results:

- In your solution PDF file (hw1.pdf), please write (1) the **explanation** about your algorithm and (2) the program's **elapsed time** (i.e., the actual time taken from the start of your program to the end). You should give an explanation of the design choices in your Spark program. If your design choice and explanation are convincing, you can get full credit.

- Your source code in one file (with file name `hw1_1.py`).
  Your code should **print** the top-10 user pairs with their counts (total 10 lines). The printed results must be sorted by their counts in descending order, as following format for each line:
  `<User><TAB><User><TAB><Count>`
  You should use the `print` built-in function in Python to produce outputs. See the description above for more detailed information about output format.

2 Frequent Itemsets (30 pts)

(a) [10 pts] Solve the following problem from the MMDS textbook.

- Exercise 6.2.7
  Suppose we have market baskets that satisfy the following assumptions:
  (1) The support threshold is 10,000.
  (2) There are one million kinds of items, represented by the integers 0, 1, ..., 999999.
  (3) There are N frequent items, that is, items that occur 10,000 times or more.
  (4) There are one million pairs that occur 10,000 times or more.
  (5) There are $2M$ pairs that occur exactly once. Of these pairs, M consist of two frequent items; the other M each have at least one nonfrequent item.
  (6) No other pairs occur at all.
  (7) Integers are always represented by 4 bytes.

  Suppose we run the A-Priori Algorithm and can choose on the second pass between the triangular-matrix method for counting candidate pairs and a hash table of item-item-count triples. Neglect in the first case the space needed to translate between original item numbers and numbers for the frequent items, and in the second case neglect the space needed for the hash table. As a function of N and M, what is the minimum number of bytes of main memory needed to execute the A-Priori Algorithm on this data?

(b) [20 pts] Find frequent itemsets and association rules using the A-Priori algorithm.

Suppose you are an online retailer like Amazon and want to improve the shopping experience by analyzing customer behavior. In this problem, you will find **frequent item pairs** and then generate strong association rules from them.

Download `browsing.txt`[3] from eTL and use it as a- dataset. Each line represents a browsing session of a customer where item ID strings are delimited by spaces.

Your implementation should consist of two main stages:

(a) First, find all frequent item pairs with a **support threshold of 100**. To store counts of pairs, please use the triangular method (not the triples method).

(b) Second, based on the frequent pairs and frequent items found in the first stage, generate all **association rules** that have a **confidence of 50% or higher**. The confidence of a rule $A \to B$ is calculated as:

$$\text{Confidence}(A \to B) = \frac{\text{Support}(\{A, B\})}{\text{Support}(\{A\})}$$

Your code (`hw1_2_b.py`) should `print` total 12 lines: the number of frequent pairs, the number of valid association rules found that exceed the confidence threshold, and the top-10 strongest association rules, which must be sorted by their confidence in descending order, as the following format:

```
<number of frequent pairs>
<number of valid association rules (confidence >= 50%)>
Rule: {<A>} -> {<B>}, Confidence: <conf(A -> B)>, Support: <supp({A, B})>
... (10 lines total for rules)
Rule: {<X>} -> {<Y>}, Confidence: <conf(X -> Y)>, Support: <supp({X, Y}>
```

Example (one rule line):

```
Rule:  DAI93865 -> FRO40251, Confidence:  1.000000, Support:  208
```

where the confidence value should be represented as a floating-point number (e.g., 0.65). For any ties in confidence, please sort by support in descending order.

This problem does not require Spark programming, and your code does not have to be long. **Do not use Spark!**

Please **use command-line** arguments to obtain the file path of the dataset. (Do not fix the path in your code.) For example, run:

```
python hw1_2_b.py YourPathTo/browsing.txt
```

Please submit the following results:

- The program's elapsed time (i.e., the actual time taken from the start of your program to the end) on the dataset. Please write it in your solution pdf file (hw1.pdf).
- Your source code in one file (with file name `hw1_2_b.py`).
- A text output produced by your program (exactly 12 lines as specified above).

[3] This dataset is from the Stanford Large Network Dataset Collection.

3 Finding Similar Items (40 pts)

(a) [10 pts] Solve the following problem from the MMDS textbook.

- Exercise 3.6.1
  What is the effect on probability of starting with the family of minhash functions and applying:

  (a) A 2-way AND construction followed by a 3-way OR construction.
  (b) A 3-way OR construction followed by a 2-way AND construction.
  (c) A 2-way AND construction followed by a 2-way OR construction, followed by a 2-way AND construction.
  (d) A 2-way OR construction followed by a 2-way AND construction, followed by a 2-way OR construction followed by a 2-way AND construction.

(b) [20 pts] Find similar documents using minhash-based LSH.

Suppose you are looking for very similar articles within a large article set. In this problem, implement the minhash-based LSH algorithm in Chapter 3.4.3 to efficiently find articles that have high Jaccard similarities.

Before you solve this problem, you should understand the concept of minhash-based LSH. Please read carefully the textbook and the lecture notes.

Download `articles.txt`[4] from eTL and use it as the dataset. Each line of the file is in the following format:

```
<ARTICLE ID><SPACE><TEXT>
```

where `<ARTICLE ID>` is the article ID, `<SPACE>` is a single space, and `<TEXT>` is the text of the article.

When extracting k-shingles,

- Consider a shingle unit as an alphabetic character
- Treat the white space as a character
- Ignore non-alphabet characters except the white space
- Convert all characters to lower case
- Extract 3-shingles (i.e., set k=3)

For consistency across implementations, construct the global list of all unique 3-shingles and sort it **lexicographically** before assigning row indices for MinHashing. This ensures deterministic shingle-to-row mappings and consistent MinHash signatures.

[4] This dataset is from the University of Edinburgh

When generating random hash functions, use the form $(ax + b)\%c$ as described in the textbook. Let n be the number of rows, as in Figure 3.4. Set c to the smallest prime number greater than or equal to n. Then choose a uniformly at random from $[0, c-1]$, and choose b independently in the same range. To ensure reproducibility, set **np.random.seed(0)**.

Set b and r so that the threshold is about 0.9. You can set $b = 6$ and $r = 20$, unless you prefer a different setting.

Your code (`hw1_3_b.py`) should `print` the candidate article ID pairs whose signature components agree by at least 0.9, as following format for each line:

```
<ARTICLE ID><TAB><ARTICLE ID><TAB><SIMILARITY>
```

Example (one line):

```
t269<TAB>t8413<TAB>1.000000
```

where the order between the first user and the second user is not important. You can get full credit if your output contains all correct pairs regardless of the order between lines.

This problem does not require Spark programming. (**Do not use Spark**)

Please **use command-line** arguments to obtain the file path of the dataset. (Do not fix the path in your code.) For example, run:

```
python hw1_3_b.py YourPathTo/articles.txt
```

Please submit the following results:

- The program's elapsed time (i.e., the actual time taken from the start of your program to the end) on the dataset. Please write it in your solution pdf file (hw1.pdf).
- Your source code in one file (with file name `hw1_3_b.py`).
  Your code should **print** the candidate article ID pairs. Please use the `print` built-in function in Python to produce outputs. See the description above for more detailed information about output format.

(c) [10 pts] Find similar documents using TF-IDF and the cosine distance.

In this problem, you will find similar documents from the same dataset as in (b), but with the TF-IDF-based approach and cosine similarities.

Before you solve this problem, you should understand the concepts of TF-IDF and the cosine distance. Please read carefully the textbook and the lecture notes. Refer to Section 1.3.1 for TF-IDF, and 3.5.4 and 3.7.2 for the cosine distance.

In this problem, you will:

- Compute the TF-IDF vector for each document.
- Use *LSH with random hyperplanes* to identify candidate document pairs.
- Calculate the cosine distance between the TF-IDFs of candidate pairs.
- Identify pairs of documents whose cosine distance is below a certain threshold, which indicates high similarity.

When generating the TF-IDF vectors, you will:

- Use the number of times a term appears in a document as the term frequency.
- Use $\text{IDF}(t) = \log(\frac{N}{df(t)})$, where N is the total number of documents, and $df(t)$ is the number of documents having term t, as the inverse document frequency.

When constructing TF-IDF vectors, build the global vocabulary consisting of all unique terms in the dataset and sort it **lexicographically** before assigning vector dimensions. This ensures that all documents use a consistent term-to-dimension mapping.

When applying LSH, you will:

- Use **10 random hyperplanes** to hash the TF-IDF vectors into buckets. The goal is to group similar documents into the same bucket with high probability.
- Use `np.random.seed(0)` to ensure the random hyperplanes are generated deterministically. This will allow you to get consistent results on every run.
- For each document, compute the hash value by determining on which side of the hyperplanes the document vector lies (positive or negative side).
- Only compare documents within the same bucket to calculate the distance.

When calculating the cosine distance, you will:

- Compute $1 - \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$ where $\vec{A}$ and $\vec{B}$ are the TF-IDF vectors of two documents.
- Consider document pairs with the cosine distance below 0.1 as "similar".

Your code (`hw1_3_c.py`) should `print` all candidate article ID pairs whose cosine distance is below the threshold of 0.1, as following format for each line:

```
<ARTICLE ID><TAB><ARTICLE ID><TAB><COSINE SIMILARITY>
```

Example (one line):

```
t1621<TAB>t7958<TAB>1.000000
```

The orders (a) between the first and the second article at each line and (b) between different lines are not important. You will get full credit if your output contains all correct pairs regardless of the order.

This problem does not require Spark programming; **do not use Spark!**

Please use **command-line arguments** to obtain the file path of the dataset. Please do not hard-code the path in your code. For example, run:

```
python hw1_3_c.py YourPathTo/articles.txt
```

Please submit the following results:

- The program's elapsed time (i.e., the actual time taken from the start of your program to the end). Please write it in your solution pdf file (hw1.pdf).
- Your source code in one file (with file name `hw1_3_c.py`).
- **print** all candidate article ID pairs whose cosine distance is below the threshold of 0.1, along with their cosine similarity scores. Please use the `print` built-in function in Python to produce outputs. See the description above for more detailed information about output format.
- Make sure to set `np.random.seed(0)` when generating the random hyperplanes, so the result is deterministic. By comparing your results to the provided examples, you can verify that your implementation is correct.

Answers to Frequently Asked Questions

1. About the program's **elapsed time**: If you well-designed the codes, the programs' elapsed times (for all problems in HW1) should be less than 10 minutes. However, TAs will give 30 minutes when evaluating students' codes. If your code runs more than 30 minutes, TAs will not give the credits.
2. About **design choices**: You have to decide on your own which design choices including data structures (e.g., list, array, and dictionary) are good for each program. There can be multiple correct choices, and TAs will not answer which design choices are optimal or acceptable.
3. In hw1_1, you can implement a function via def or lambda.
4. In hw1_1, you can assume that the given adjacency list is undirected, i.e., if (A,B) exists, then (B,A) also exists.
5. In hw1_1, if you still have memory error with --driver-memory 8G command, you can use provided DM01 server or try to remove useless(not related to result) "print" statement in your code.
6. Please use the `print` **built-in function in Python** to produce outputs.
7. Also, note that `<TAB>` means `\t` in the Python print function.
8. In this homework, you do not need to save the output files.
9. When you need to test your code: Our recommendation is to create a 'small' test file that you can check the answers on your own.